МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТк

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность ИИ26(з)

Выполнил
А.В. Семёнов,
студент группы ИИ-26

Проверил
К.В. Андренко,
ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2^n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**
   - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
   - В качестве функции активации использовать сигмоиду
   - Провести **два эксперимента** по обучению:
     А. С фиксированным шагом обучения α
     Б. С адаптивным шагом
   - Выход сети интерпретировать как вероятность принадлежности к классу «1».
   - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. **Провести анализ сходимости:**
   - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
   - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
   - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях α
4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**
   - Программа должна принимать на вход вектор из n чисел (0 или 1).
   - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).

Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

16	8	AND^i

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

X_raw = np.array([[3.0, 4.0], [-3.0, 4.0], [3.0, -4.0], [-3.0, -4.0]], dtype=float)
E = np.array([1.0, 1.0, 0.0, 0.0], dtype=float)

def sigmoid(s):
    if s >= 0:
        z = np.exp(-s)
        return 1.0 / (1.0 + z)
    z = np.exp(s)
    return z / (1.0 + z)
```

```
def bce_sum(y, e, eps=1e-12):
    y = np.clip(y, eps, 1.0 - eps)
    return float(-np.sum(e * np.log(y) + (1.0 - e) * np.log(1.0 - y)))

def mse_mean(y, e):
    diff = np.clip(e - y, -1e6, 1e6)
    return float(np.mean(diff * diff))

def sse_sum(y, e):
    y = np.asarray(y, dtype=float).reshape(-1)
    e = np.asarray(e, dtype=float).reshape(-1)
    return float(np.sum((y - e) ** 2))

def alpha_adaptive(x):
    return 1.0 / (1.0 + float(np.sum(x ** 2)))

class LR1Net:
    def __init__(self, lr=0.1, seed=42, w_clip=50.0):
        rng = np.random.default_rng(seed)
        self.w = rng.uniform(-0.5, 0.5, size=2).astype(float)
        self.b = float(rng.uniform(-0.5, 0.5))
        self.lr = float(lr)
        self.w_clip = float(w_clip)

    def forward_linear(self, x):
        return float(np.dot(self.w, x) + self.b)

    def train_epoch(self, X, E, shuffle=True, seed=0):
        idx = np.arange(len(X))
        if shuffle:
            rng = np.random.default_rng(seed)
            rng.shuffle(idx)
        for i in idx:
            x = X[i]
            e = E[i]
            y = self.forward_linear(x)
            err = e - y
            self.w += self.lr * err * x
            self.b += self.lr * err
            self.w = np.clip(self.w, -self.w_clip, self.w_clip)
            self.b = float(np.clip(self.b, -self.w_clip, self.w_clip))
        y_all = np.array([self.forward_linear(x) for x in X], dtype=float)
        return mse_mean(y_all, E)

def run_lr1_mse_fixed():
    x_scale = np.max(np.abs(X_raw), axis=0)
    X = X_raw / x_scale
```

```
    learning_rates = [0.01, 0.05, 0.1, 0.2, 0.5, 0.8]
    max_epochs = 100
    histories = {}
    best_lr = None
    best_final = float("inf")
    for lr in learning_rates:
        model = LR1Net(lr=lr, seed=42, w_clip=50.0)
        hist = []
        for ep in range(max_epochs):
            cur = model.train_epoch(X, E, shuffle=True, seed=1000 + ep)
            cur_es = cur * len(E)
            hist.append(cur_es)
            if cur_es < 1e-8 * len(E):
                break
        histories[lr] = np.array(hist, dtype=float)
        if hist[-1] < best_final:
            best_final = hist[-1]
            best_lr = lr
    model = LR1Net(lr=best_lr, seed=42, w_clip=50.0)
    for ep in range(len(histories[best_lr])):
        model.train_epoch(X, E, shuffle=True, seed=1000 + ep)
    return histories[best_lr], best_lr, x_scale, model.w.copy(), float(model.b)

class LR2Net:
    def __init__(self, seed=42, w_clip=50.0):
        rng = np.random.default_rng(seed)
        self.w = rng.uniform(-0.5, 0.5, size=(2,))
        self.T = 0.0
        self.w_clip = float(w_clip)

    def forward(self, x):
        return float(np.dot(self.w, x) - self.T)

    def update_delta_rule(self, x, e, alpha):
        y = self.forward(x)
        err = (y - e)
        self.w = self.w - alpha * err * x
        self.T = self.T + alpha * err
        self.w = np.clip(self.w, -self.w_clip, self.w_clip)
        self.T = float(np.clip(self.T, -self.w_clip, self.w_clip))

def run_lr2_mse_adaptive(alpha_fixed=0.1, Ee=1e-6, max_epochs=2000, seed=123):
    x_scale = float(np.max(np.abs(X_raw)))
    X = X_raw / x_scale
    model = LR2Net(seed=42, w_clip=50.0)
    rng = np.random.default_rng(seed)
    hist = []
    n = X.shape[0]
```

```
    for _ in range(max_epochs):
        idx = np.arange(n)
        rng.shuffle(idx)
        for i in idx:
            x = X[i]
            e = E[i]
            alpha = alpha_adaptive(x)
            model.update_delta_rule(x, e, alpha)
        y_all = np.array([model.forward(x) for x in X], dtype=float)
        Es = sse_sum(y_all, E)
        hist.append(Es)
        if Es <= Ee:
            break
    return np.array(hist, dtype=float)

class SigmoidNet:
    def __init__(self, seed=42, w_clip=50.0):
        rng = np.random.default_rng(seed)
        self.w = rng.uniform(-0.5, 0.5, size=(2,))
        self.T = 0.0
        self.w_clip = float(w_clip)

    def s(self, x):
        return float(np.dot(self.w, x) - self.T)

    def y(self, x):
        return float(sigmoid(self.s(x)))

    def update_bce(self, x, e, alpha):
        y = self.y(x)
        err = (y - e)
        self.w = self.w - alpha * err * x
        self.T = self.T + alpha * err
        self.w = np.clip(self.w, -self.w_clip, self.w_clip)
        self.T = float(np.clip(self.T, -self.w_clip, self.w_clip))

def train_bce(mode, alpha_fixed, Ee, max_epochs=5000, seed=123):
    x_scale = np.max(np.abs(X_raw), axis=0)
    X = X_raw / x_scale
    model = SigmoidNet(seed=42, w_clip=50.0)
    rng = np.random.default_rng(seed)
    hist = []
    n = X.shape[0]
    for _ in range(max_epochs):
        idx = np.arange(n)
        rng.shuffle(idx)
        for i in idx:
            x = X[i]
```

```
            e = float(E[i])
            alpha = float(alpha_fixed) if mode == "fixed" else alpha_adaptive(x)
            model.update_bce(x, e, alpha)
        y_all = np.array([model.y(x) for x in X], dtype=float)
        Es = bce_sum(y_all, E)
        hist.append(Es)
        if Es <= Ee:
            break
    return model, np.array(hist, dtype=float), x_scale

def boundary_raw_from_bce(model, x_scale_vec, x1_vals):
    w1, w2 = model.w
    T = model.T
    xs1, xs2 = x_scale_vec
    if abs(w2) < 1e-12:
        return None
    return xs2 * (T - w1 * (x1_vals / xs1)) / w2

def boundary_raw_from_lr1(best_w, best_b, x_scale_vec, x1_vals, threshold=0.5):
    a1 = best_w[0] / x_scale_vec[0]
    a2 = best_w[1] / x_scale_vec[1]
    if abs(a2) < 1e-12:
        return None
    return (threshold - best_b - a1 * x1_vals) / a2

def main():
    Ee = 1e-6
    alpha_best = 0.2

    hist_mse_fixed, best_lr, x_scale_lr1, w_lr1, b_lr1 = run_lr1_mse_fixed()
    hist_mse_adapt = run_lr2_mse_adaptive(alpha_fixed=0.1, Ee=Ee, max_epochs=2000)

    model_bce_fixed, hist_bce_fixed, x_scale_bce = train_bce("fixed", alpha_best, Ee)
    model_bce_adapt, hist_bce_adapt, _ = train_bce("adaptive", alpha_best, Ee)

    labels = [
        f"MSE + фикс (lr={best_lr})",
        "MSE + адапт",
        "BCE + фикс ",
        "BCE + адапт "
    ]
    hists = [hist_mse_fixed, hist_mse_adapt, hist_bce_fixed, hist_bce_adapt]

    plt.figure(figsize=(10, 6))
    for h, lbl in zip(hists, labels):
        plt.plot(np.arange(1, len(h) + 1), h, label=lbl, linewidth=1.8)
    plt.xlabel("Эпоха p")
    plt.ylabel("Es(p)")
```

```
plt.title("Сходимость Es(p) для 4 конфигураций")
plt.grid(True, alpha=0.35)
plt.legend()
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(9, 7))
X0 = X_raw[E == 0]
X1 = X_raw[E == 1]
ax.scatter(X0[:, 0], X0[:, 1], s=70, alpha=0.8, label="Класс 0")
ax.scatter(X1[:, 0], X1[:, 1], s=70, alpha=0.8, label="Класс 1")

x1_min, x1_max = X_raw[:, 0].min() - 2, X_raw[:, 0].max() + 2
x1_vals = np.linspace(x1_min, x1_max, 200)

x2_mse = boundary_raw_from_lr1(w_lr1, b_lr1, x_scale_lr1, x1_vals, threshold=0.5)
if x2_mse is not None:
    ax.plot(x1_vals, x2_mse, linewidth=2.2, label="Линия MSE")

x2_bce = boundary_raw_from_bce(model_bce_adapt, x_scale_bce, x1_vals)
if x2_bce is not None:
    ax.plot(x1_vals, x2_bce, linewidth=2.2, linestyle="--", label="BCE (адапт. шаг)")

ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Точки и разделяющие линии")
ax.grid(True, alpha=0.35)
ax.legend()
plt.tight_layout()
plt.show()

print(f"\n{'Конфигурация':<30} {'Эпох':>6} {'Финал Es':>14}")
print("-" * 52)
for name, hist in zip(labels, hists):
    print(f"{name:<30} {len(hist):>6} {hist[-1]:>14.6e}")

threshold = 0.5
user_points = []
user_probs = []

while True:
    s = input("\nx1 x2 > ").strip()
    if s.lower() in ("q", "quit", "exit"):
        break
    try:
        x1s, x2s = s.replace(",", " ").split()
        x_user_raw = np.array([float(x1s), float(x2s)], dtype=float)
    except Exception:
```

```
        print("Ошыбка: введи два числа через пробел или q")
        continue

    x_user = x_user_raw / x_scale_bce
    p = model_bce_adapt.y(x_user)
    cls = 1 if p >= threshold else 0
    print(f"y={p:.6f} class={cls}")

    user_points.append([x_user_raw[0], x_user_raw[1]])
    user_probs.append(p)

    fig, ax = plt.subplots(figsize=(9, 7))
    ax.scatter(X0[:, 0], X0[:, 1], s=70, alpha=0.8, label="Класс 0")
    ax.scatter(X1[:, 0], X1[:, 1], s=70, alpha=0.8, label="Класс 1")
    up = np.array(user_points, dtype=float)
    ax.scatter(up[:, 0], up[:, 1], marker="x", s=140, linewidths=2.5, label="Введённые
точки")
    for (x1p, x2p), pp in zip(user_points, user_probs):
        ax.text(x1p + 0.1, x2p + 0.1, f"y={pp:.3f}", fontsize=9)

    x2_mse = boundary_raw_from_lr1(w_lr1, b_lr1, x_scale_lr1, x1_vals, threshold=0.5)
    if x2_mse is not None:
        ax.plot(x1_vals, x2_mse, linewidth=2.2, label="Линия MSE")
    x2_bce = boundary_raw_from_bce(model_bce_adapt, x_scale_bce, x1_vals)
    if x2_bce is not None:
        ax.plot(x1_vals, x2_bce, linewidth=2.2, linestyle="--", label="BCE (адапт. шаг)")

    ax.set_xlabel("x1")
    ax.set_ylabel("x2")
    ax.set_title("Режым функцианирования")
    ax.grid(True, alpha=0.35)
    ax.legend()
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    main()
```

Результат:

Всего наборов: 256
Единиц на выходе: 255
Нулей на выходе: 1

Train: 205
Test: 51

Эксперимент A — фиксированный шаг

Запуск обучения для alpha = 0.01

Запуск обучения для alpha = 0.05

Запуск обучения для alpha = 0.1

Запуск обучения для alpha = 0.2

Запуск обучения для alpha = 0.5

Таблица результатов экспериментов
alpha | epochs | BCE_train | BCE_test | acc_train | acc_test
0.0100 | 3000 | 2.072464 | 0.586546 | 0.9951 | 1.0000
0.0500 | 3000 | 0.653319 | 0.855598 | 1.0000 | 1.0000
0.1000 | 3000 | 0.362378 | 0.953723 | 1.0000 | 1.0000
0.2000 | 3000 | 0.192322 | 1.015974 | 1.0000 | 1.0000
0.5000 | 3000 | 0.079317 | 1.058527 | 1.0000 | 1.0000

Лучшая fixed-модель:
alpha = 0.01
epochs = 3000

Параметры модели
w1 = -1.498166
w2 = -1.881211
w3 = -1.434538
w4 = -1.759576
w5 = -1.731090
w6 = -1.423852
w7 = -1.784695
w8 = -1.349583
Порог T = -13.662366

Точность на полной таблице: 0.996094
Количество ошибок: 1
X = [1 1 1 1 1 1 1 1], y_true = 0, y_pred = 1, p = 0.689900

Эксперимент B — адаптивный шаг
Эпоха 1 | BCE_train = 10.307723 | BCE_test = 0.355318
Эпоха 200 | BCE_train = 2.175067 | BCE_test = 0.445028
Эпоха 400 | BCE_train = 1.389540 | BCE_test = 0.637750
Эпоха 600 | BCE_train = 1.060107 | BCE_test = 0.609614
Эпоха 800 | BCE_train = 0.855972 | BCE_test = 0.748738
Эпоха 1000 | BCE_train = 0.725698 | BCE_test = 0.741100
Эпоха 1200 | BCE_train = 0.631295 | BCE_test = 0.768040
Эпоха 1400 | BCE_train = 0.560395 | BCE_test = 0.774392
Эпоха 1600 | BCE_train = 0.503196 | BCE_test = 0.827994
Эпоха 1800 | BCE_train = 0.458254 | BCE_test = 0.820252

Эпоха 2000 | BCE_train = 0.421252 | BCE_test = 0.814391
Эпоха 2200 | BCE_train = 0.389936 | BCE_test = 0.815468
Эпоха 2400 | BCE_train = 0.362589 | BCE_test = 0.831903
Эпоха 2600 | BCE_train = 0.338900 | BCE_test = 0.850786
Эпоха 2800 | BCE_train = 0.318668 | BCE_test = 0.849868
Эпоха 3000 | BCE_train = 0.300592 | BCE_test = 0.858238

Параметры модели
w1 = -3.312435
w2 = -5.532523
w3 = -3.311751
w4 = -5.487557
w5 = -5.525383
w6 = -3.235175
w7 = -5.529491
w8 = -3.183373
Порог T = -33.313743

Точность на полной таблице: 1.000000
Ошибок нет

Сравнение моделей
Fixed full accuracy = 0.996094
Adaptive full accuracy = 1.000000
Fixed test BCE = 0.586546
Adaptive test BCE = 0.858238

Для режима функционирования выбрана модель: адаптивный шаг

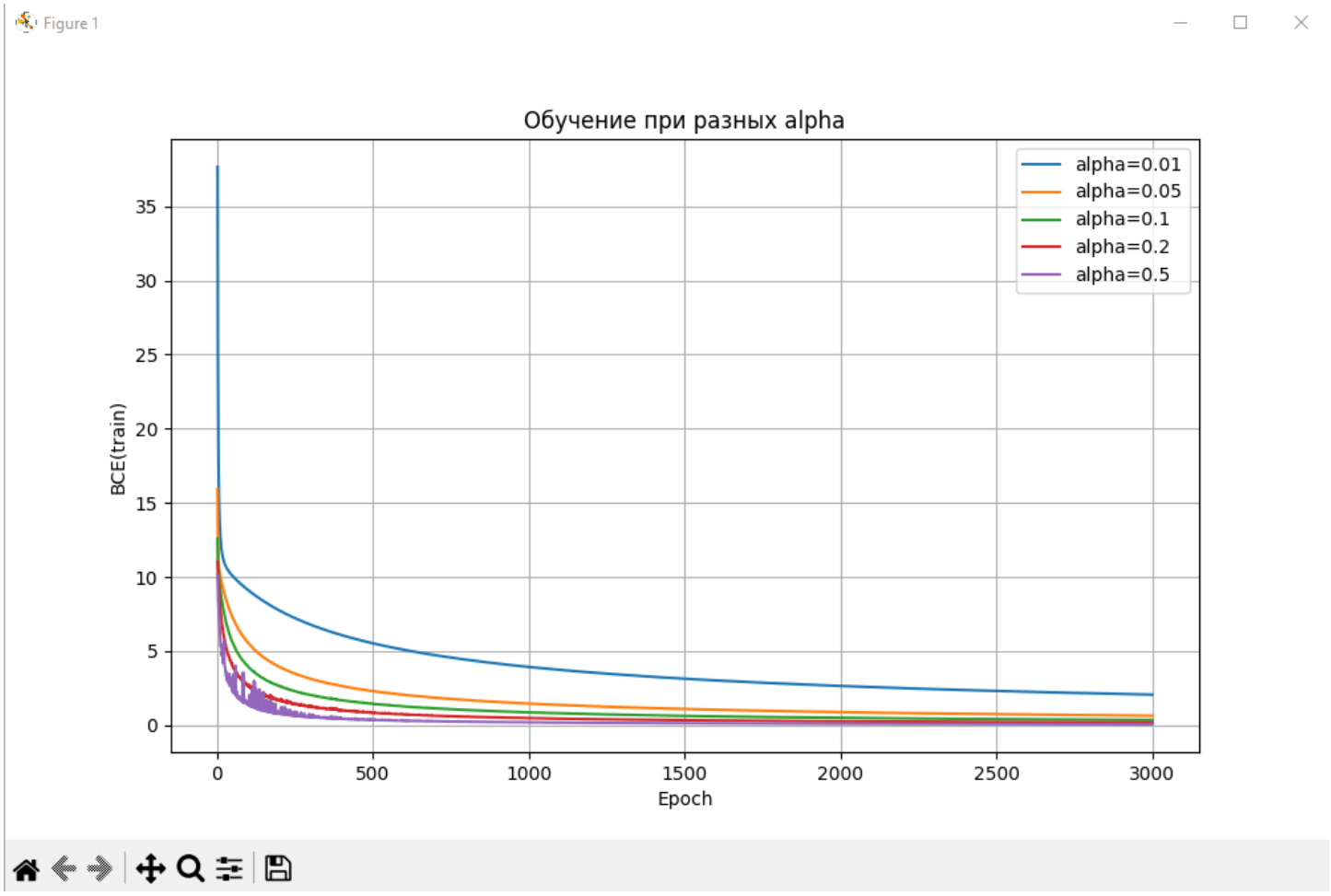
Режим функционирования
Введите 8 чисел (0 или 1)
Для выхода нажмите q
x1 x2 x3 x4 x5 x6 x7 x8 > 1 1 1 1 1 1 1 1
Вероятность = 0.141371
Класс = 0
x1 x2 x3 x4 x5 x6 x7 x8 > 0 0 0 0 0 0 0 0
Вероятность = 1.000000
Класс = 1
x1 x2 x3 x4 x5 x6 x7 x8 > 1 1 1 1 1 1 1 0
Вероятность = 0.798899
Класс = 1
x1 x2 x3 x4 x5 x6 x7 x8 > 1 0 1 0 1 0 1 0
Вероятность = 1.000000
Класс = 1
x1 x2 x3 x4 x5 x6 x7 x8 > 0 1 0 1 0 1 0 1
Вероятность = 1.000000
Класс = 1
x1 x2 x3 x4 x5 x6 x7 x8 > 1 1 1 1 0 0 0 0

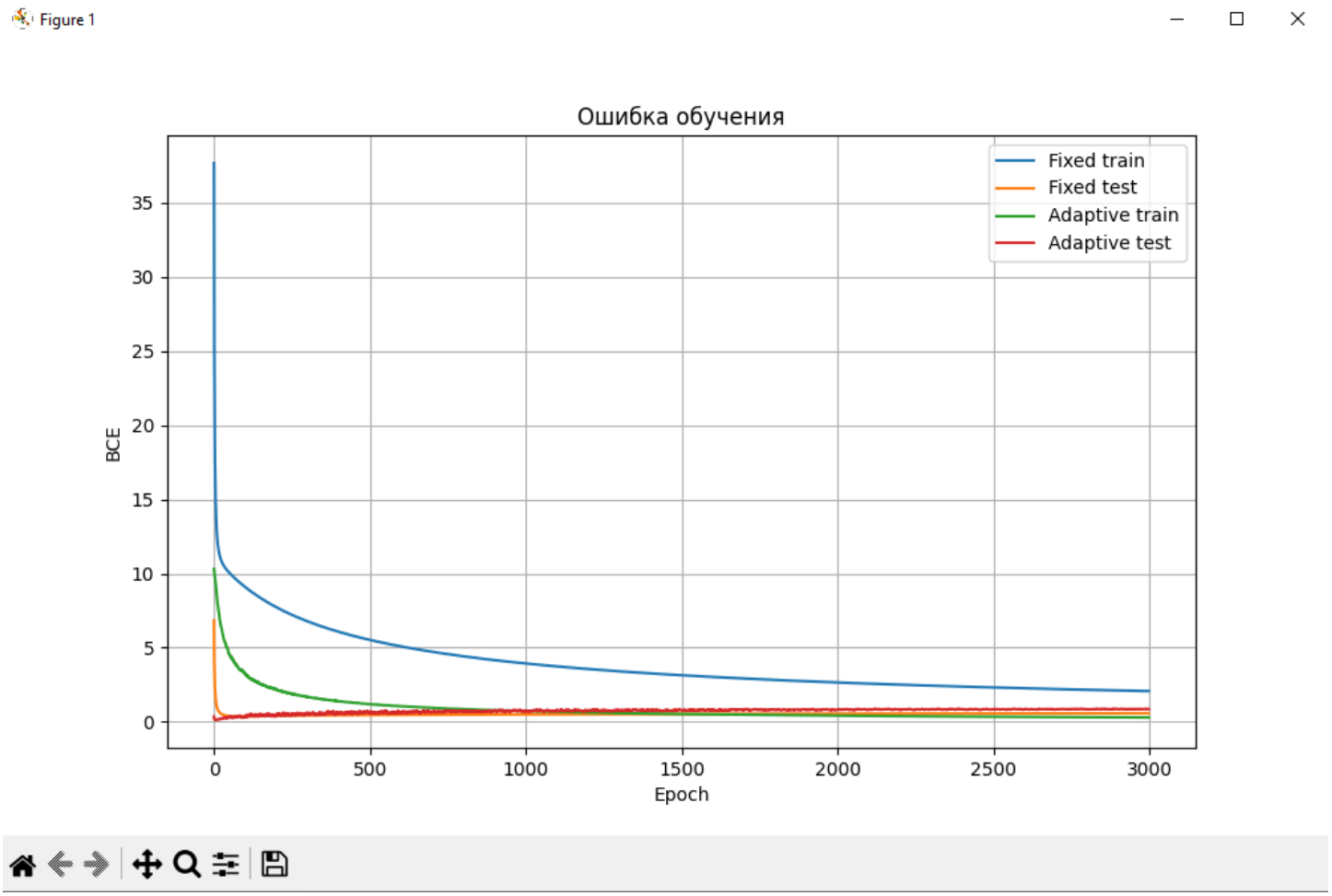
Вероятность = 1.000000
Класс = 1
x1 x2 x3 x4 x5 x6 x7 x8 >

Анализ результатов

Сигмоидальный однослойный персептрон успешно восстановил функцию ANDi по обучающей выборке и корректно воспроизвёл полную таблицу истинности. Это подтверждает способность персептрона представлять базовые логические операции.

Рисунки с результатами работы программы





Вывод: : Реализовал нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнил эффективность обучения при различных начальных условиях и параметрах шага.